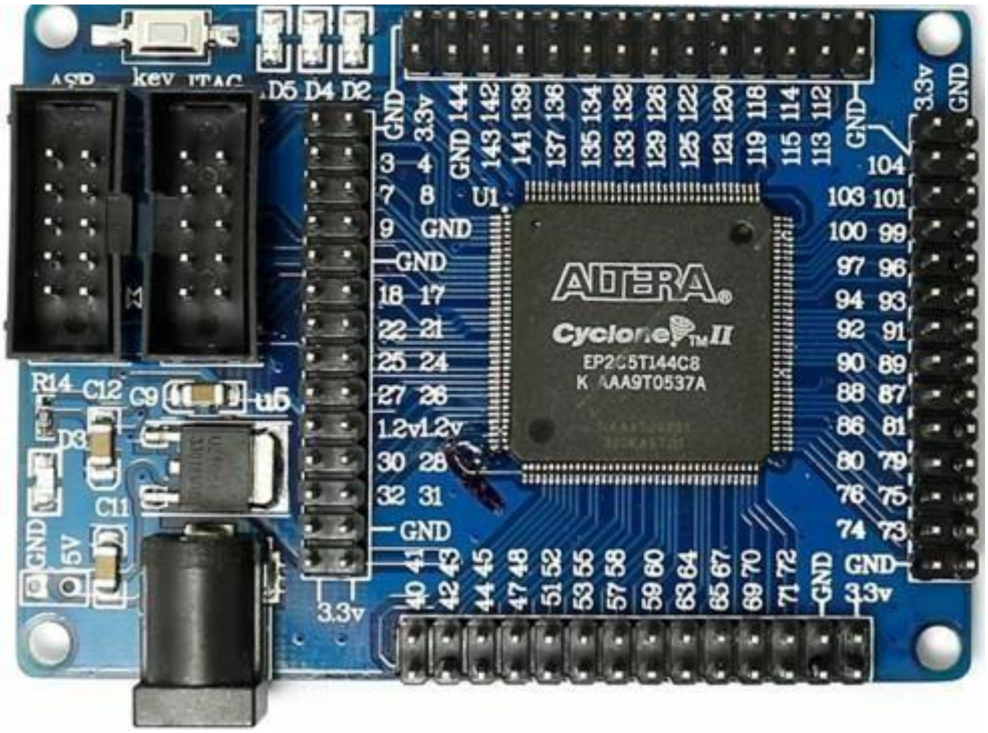
Experiment 1

Introduction to FPGA Architecture and Design Methodologies



Objective:

This course aims to give an understanding of FPGA architecture with a special focus on the Altera Cyclone II EP2C5T144C8 FPGA. The experiment will examine how FPGAs have changed over time give a look, at FPGA architecture and show all design methods used in FPGA-based digital design. Students will learn the skills needed to create and implement advanced FPGA designs.

1. Introduction to FPGAs:

1.1. What is an FPGA?

1.2. ● A Field-Programmable Gate Array (FPGA) is a

emiconductor device that uses a grid of logic blocks (CLBs) linked

by programmable interconnects. FPGA devices are

reconfigurable which means the logic inside an FPGA can be

reprogrammed to run designs or algorithms after the device is

made. This reconfigurability is different from fixed-function

Application-Specific Integrated Circuits (ASICs). Because of this

FPGA devices are very flexible for prototyping and for use, in

different applications.

1.3. Evolution of FPGA Technology: From PLAs to Modern FPGAs

Origins of Programmable Logic:

- The foundation of FPGA technology began with Programmable Logic Arrays (PLAs) and Programmable Array Logic (PALs) in the 1970s. PLAs enabled the creation of logic functions using programmable AND and OR gates but complexity was limited. PALs made the design process easier by providing a fixed OR array and a programmable AND array which increased efficiency and usability. Scalability remained limited.

CPLDs as a Bridge to FPGAs

- As digital designs became more complex, **Complex Programmable Logic Devices (CPLDs)** were introduced to overcome some of the limitations of earlier PLDs. A CPLD combines several PAL-like logic blocks on a single chip and connects them using a programmable interconnection matrix. This increased the available logic capacity and provided greater flexibility compared with traditional PLDs. However, CPLDs were still not powerful enough for very large and highly complex digital designs.

Development of FPGAs

- The limitations of PLAs, PALs, and CPLDs eventually led to the development of **Field-Programmable Gate Arrays (FPGAs)** during the mid-1980s. In 1985, Xilinx introduced the first commercial FPGA, marking an important step in programmable digital technology. FPGAs use **Configurable Logic Blocks (CLBs)** that contain logic elements, flip-flops, and programmable connections. These components can be configured in different ways to implement a wide variety of digital circuits. The flexible interconnection between the logic blocks allows FPGAs to handle larger and more complicated designs while remaining highly adaptable.

Altera's Contributions

- **Altera**, founded in 1983, quickly became one of the important companies in the FPGA industry. Its early developments, including the EPROM-based EP300, helped establish the foundation for its later success. One of Altera's notable product families was the **Cyclone series**, which was designed to provide a good combination of cost, performance, and power efficiency. The Cyclone II series, in particular, became useful for both educational projects and commercial applications. The **Cyclone II EP2C5T144C8 FPGA** is a good example of this balance, offering 4,608 Logic Elements along with embedded memory blocks and dedicated multipliers. These features make it a practical and reliable platform for learning and developing digital systems.

1.2. Importance of FPGAs in Modern Electronics

- FPGAs have become an important part of modern electronic systems because they provide a flexible way to implement complex digital circuits. Unlike fixed hardware, an FPGA can be reconfigured to meet changing design requirements, even after the system has been deployed. This is especially useful when quick development and shorter time-to-market are important.
- Today, FPGAs are used in many areas, including **telecommunications, signal processing, automotive electronics, aerospace, artificial intelligence, and machine learning**. Their ability to perform multiple operations in parallel makes them well suited for real-time processing and high-throughput applications. As a result, FPGAs are widely used in situations where both high performance and flexibility are required.

2. Detailed Overview of Altera Cyclone II EP2C5T144C8

2.1. Architectural Overview

- The **Altera Cyclone II EP2C5T144C8 FPGA** is a low-cost device designed to provide a good balance between logic capacity, processing speed, and power consumption. It is manufactured using **90 nm**

process technology, which helps maintain relatively efficient power usage while providing a useful set of logic and memory resources.

- The Cyclone II family was developed to support a wide range of digital applications. It can be used for basic logic circuits as well as more advanced and complex system designs. Its combination of programmable logic, memory resources, and other dedicated hardware features makes it a suitable choice for applications ranging from academic projects to practical digital-system development.

2. Detailed Overview of Altera Cyclone II EP2C5T144C8

2.1. Architectural Overview

- The **Altera Cyclone II EP2C5T144C8 FPGA** is a low-cost device designed to provide a good balance between logic capacity, processing speed, and power consumption. It is manufactured using **90 nm process technology**, which helps maintain relatively efficient power usage while providing a useful set of logic and memory resources.
- The Cyclone II family was developed to support a wide range of digital applications. It can be used for basic logic circuits as well as more advanced and complex system designs. Its combination of programmable logic, memory resources, and other dedicated hardware features makes it a suitable choice for applications ranging from academic projects to practical digital-system development.

1. FPGA Chip (Center of the Board)

- **Cyclone II FPGA:** The main component located at the center of the board is the **Cyclone II EP2C5T144C8 FPGA**. The chip has pin numbers marked around it, which help identify and connect the different input and output signals. It contains logic elements, embedded memory blocks, and dedicated multipliers.

These resources can be configured and connected using the FPGA's programmable routing system, allowing users to design and implement different types of digital circuits.

2. Digital GPIO (General-Purpose Input/Output)

- **GPIO Headers (Top, Right, and Bottom):** Several GPIO headers are available along the top, right, and bottom edges of the board. GPIO pins can be programmed to work either as inputs or outputs and are directly connected to the FPGA's internal logic. They can be used to connect external devices such as sensors, switches, LEDs, displays, motors, and other electronic components. Since the pins can be individually configured, the board provides a flexible way to interface the FPGA with external hardware.

3. User LEDs and Power Indicator LED (Top-Left Corner)

- **LED Indicators (D2, D3, D4, D5):** The board includes several LEDs that provide a simple way to display information during testing and operation. LEDs **D2, D4, and D5** are user-programmable and can be controlled through the FPGA design. They can be used to indicate different logic conditions, signal activity, or the status of a particular operation.
- **Power LED (D3):** LED **D3** serves as the power indicator. It lights up when the board is receiving the required power supply. This makes it easy to check whether the board is powered correctly.

4. Programming Interfaces (Left Side)

JTAG and ASP Interfaces

- **JTAG Interface:** The board provides a **JTAG (Joint Test Action Group)** interface, which is mainly used for programming, testing, and debugging the FPGA. Through JTAG, a user can transfer the FPGA configuration or bitstream to the device and perform debugging and testing during development.
- **ASP Interface:** The **ASP (Active Serial Programming)** interface provides another way to configure the FPGA. It is generally used to program the configuration memory associated with the FPGA. This allows the programmed design to be loaded again when the board is powered on.

5. Power Supply (Bottom-Left Corner)

- **DC Power Jack (5V Input):** The board receives its power through a DC power jack with a maximum input voltage of **5V**. The supplied voltage is regulated on the board and distributed to the FPGA and other components that require power. A **3.3V output** is also available on the board, which can be used to supply power to suitable external devices connected through the GPIO headers.

6. Ground Pins and 3.3V Pin

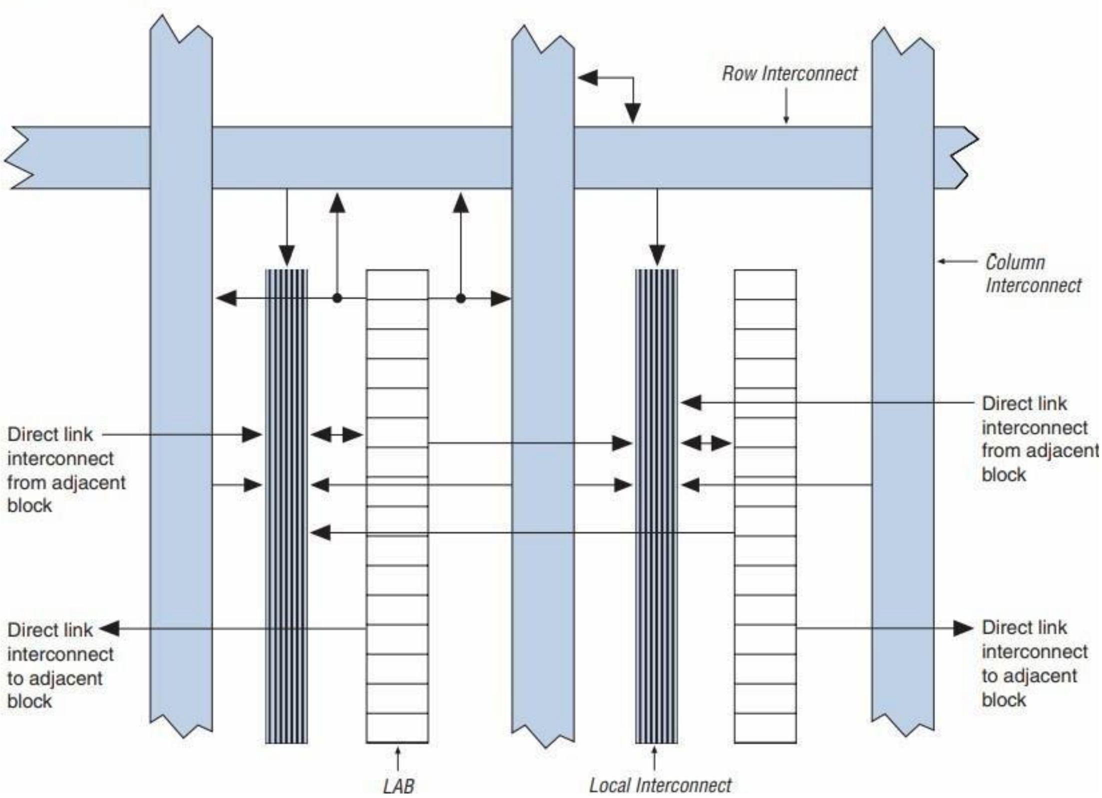
- **GND and 3.3V (VCC) Pins:** Multiple **GND (ground)** pins are provided on the board. These pins offer a common electrical reference for the FPGA and external devices connected to it. The **3.3V (VCC)** pin provides a regulated voltage supply that can be used to power compatible external components connected to the GPIO pins.

7. Key Switch

- **Key Switch:** A key switch is located near the programming interfaces. Depending on the board's design, it can be used for functions such as resetting the FPGA, entering a particular programming mode, or providing a basic digital input for testing. It can therefore be useful when experimenting with simple control functions or testing FPGA-based designs.

1.4. Logic Array Block (LAB):

Figure 2–5. Cyclone II LAB Structure



- **Overview**:
  LABs are the fundamental building units within the Cyclone-II architecture. Each LAB contains a group of Logic Elements (LEs), interconnect resources, and control logic. The LABs are distributed in a grid pattern across the FPGA fabric, providing a structured and scalable way to implement complex digital circuits.
- **LAB Structure**:
  Each LAB in the Cyclone-II FPGA comprises **16 Logic Elements (LEs)**. These LEs are interconnected and share common control signals, such as clocks and resets.
  The LAB is designed to optimize both logic and routing efficiency, allowing the FPGA to achieve high performance with low power consumption.
- **LAB Interconnect**:
  LABs are connected to each other through a versatile and programmable interconnect structure. This interconnect allows for flexible connections between

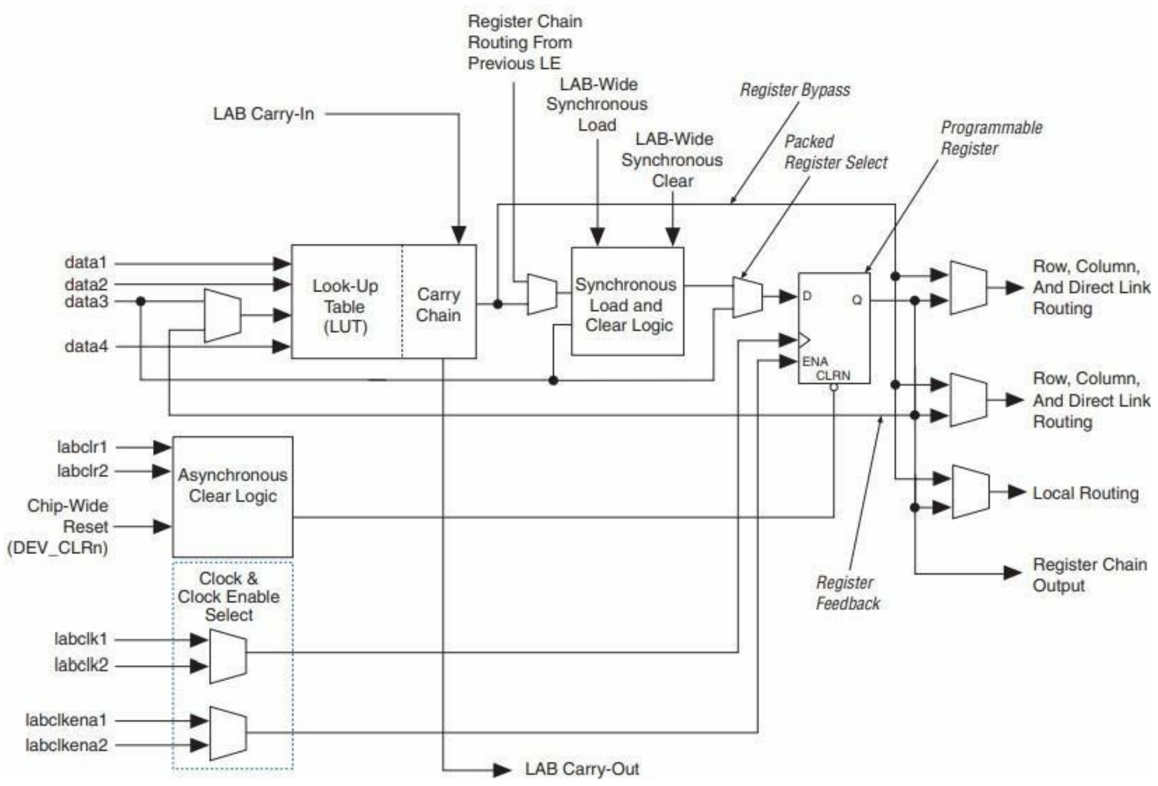
LABs and other blocks within the FPGA, such as memory blocks, DSP blocks, and I/O elements.

LABs have access to both local and global routing resources. Local routing is used for connections within or between adjacent LABs, while global routing handles connections across the FPGA, allowing for efficient implementation of wide logic functions or high-speed clock networks.

- **Control Signals**:
  LABs share control signals that manage the operation of the LEs within them. These include clock signals, synchronous and asynchronous reset signals, clock enables, and other control inputs that are essential for coordinating the timing and operation of the logic elements.

1.5. Logic Elements (LEs):

Figure 2–2. Cyclone II LE



1.1. Logic Elements (LEs)

- The basic building blocks of the **Cyclone II FPGA** are called **Logic Elements (LEs)**. Each LE is a small programmable logic unit that contains a **4-input Look-Up Table (LUT)**, a programmable flip-flop, and additional carry and cascade logic. The LUT can be used to implement any Boolean function involving up to four inputs. The flip-flop can work as either a D-type flip-flop or a latch, depending on the requirements of the design. Carry and cascade logic help in performing arithmetic operations, such as addition, and also allow signals to move efficiently between different logic elements.
- The **Cyclone II EP2C5T144C8** contains **4,608 Logic Elements**. These LEs can be configured and connected in different ways to implement both combinational and sequential circuits. The FPGA architecture is designed to make efficient use of these resources, allowing relatively large and complex digital designs to be implemented on a single chip.

1.2. Embedded Memory Blocks

- The **EP2C5T144C8** provides **13,104 bits of embedded memory**, which are organized into several **M4K memory blocks**. These memory blocks can operate in different configurations, including single-port RAM, dual-port RAM, and true dual-port RAM. They support different data widths ranging from 1 to 36 bits. This built-in memory is useful for applications such as data storage, buffering, FIFOs, and other designs that require fast access to stored data.
- Along with the embedded memory, the FPGA also provides dedicated hardware resources for arithmetic operations. These resources can be used for operations such as multiplication and **multiply-accumulate (MAC)** calculations, which are commonly required in signal-processing and other computational applications.

1.3. Clock Management and PLLs

- Proper clock management is important in FPGA designs because different parts of a circuit need to operate in a synchronized manner. The Cyclone II FPGA includes **Phase-Locked Loops (PLLs)** that help generate and control clock signals. PLLs can be used to change clock frequencies, create phase-shifted clock signals, and manage designs that use multiple clock domains.
- The FPGA also supports **clock-enable signals**. These signals can be used to control when a particular part of the circuit is active. By disabling unnecessary sections when they are not being used, clock enables can help reduce dynamic power consumption.

1.4. I/O Capabilities and Interfacing

- The Cyclone II FPGA supports several I/O standards, including **LVTTL, LVCMOS, PCI, SSTL, and HSTL**. This allows the FPGA to communicate with a wide variety of external devices, such as sensors, displays, memory devices, and communication modules.
- The **EP2C5T144C8** provides **89 user I/O pins**. These pins can be configured as inputs, outputs, or bidirectional connections depending on the requirements of the application. The I/O pins are arranged into different banks, which can operate at different voltage levels. This makes it possible to connect the FPGA to external devices that use different logic-level voltages.

2. FPGA Design Methodologies

2.1. FPGA Design Flow Overview

- FPGA development follows a systematic design process that converts an initial idea or specification into a working configuration that can be programmed onto the FPGA. The main stages of this process are described below.

Specification

- The first step is to define what the FPGA design is expected to do. This includes identifying the required functionality, performance, input/output requirements, and design constraints. This specification acts as a plan for the rest of the project.

Design Entry

- In this stage, the design is created using a **Hardware Description Language (HDL)** such as **Verilog or VHDL**. A schematic capture tool can also be used for simpler designs. HDLs are commonly preferred for larger projects because they make the design easier to modify, reuse, and manage.

Synthesis

- During synthesis, the HDL code is converted into a **gate-level netlist**, which represents the design using logic elements, gates, flip-flops, and other hardware resources. The synthesis tool also optimizes the design for the selected FPGA, taking factors such as speed, available resources, and power consumption into consideration.

Simulation

- Simulation is performed before programming the actual FPGA to check whether the design behaves as expected. **Functional simulation** verifies the logical operation of the circuit, while **timing simulation** checks how the circuit behaves with respect to timing requirements. **ModelSim** is commonly used for this purpose and provides waveform displays that make it easier to observe signals and identify problems.

Implementation

- The implementation stage places the synthesized design onto the actual FPGA architecture. During **placement**, logic elements, memory blocks, and other resources are assigned to specific locations on the chip. **Routing** then creates the required connections between these resources. The completed design can be analyzed for resource usage and timing performance.

Verification

- After implementation, the design is checked to make sure it satisfies the required timing constraints. **Static Timing Analysis (STA)** is used to identify timing problems, including setup and hold-time violations. The design can also be tested directly on the hardware using test equipment or hardware-in-the-loop techniques to confirm that it works correctly under real operating conditions.

Configuration

- The final stage is to generate the **bitstream file**, which contains all the configuration information required by the FPGA. This file is then loaded onto the FPGA and configures its logic elements, routing connections, and I/O settings. Once configuration is complete, the FPGA can perform the functionality defined by the design.

2.2. Tools and Resources

Quartus II

- **Quartus II** is Altera's main development software for FPGA design. It provides tools for entering the design, synthesizing HDL code, assigning pins, analyzing timing, and implementing the design on the FPGA. It supports both **Verilog and VHDL** and includes features that help with design optimization, compilation, and timing analysis.

ModelSim

- **ModelSim** is a simulation tool used to test FPGA designs before they are programmed onto the hardware. It supports both Verilog and VHDL and provides useful debugging features such as waveform viewing and signal tracing. This allows designers to identify and correct errors early in the development process.

3. Hands-on with Altera Cyclone II EP2C5T144C8

3.1. Setting Up the Development Environment

Hardware Setup

- The first step is to connect the **Altera Cyclone II EP2C5T144C8 development board** to a computer using a **USB-Blaster** or a compatible programming cable. The required drivers should be installed so that the computer and Quartus II can properly detect the FPGA board.

Software Installation

- Install **Quartus II** on the development computer. The appropriate device family and FPGA model should be selected during project setup. For this project, the target device is the **Cyclone II EP2C5T144C8**.

Project Creation

- Create a new project in Quartus II and select the **EP2C5T144C8** as the target FPGA. The project acts as the main workspace for the design and includes the HDL source files, pin-assignment files, and simulation testbenches required for development and testing.

3.2. Basic Design Implementation

Simple LED Example

- As a basic introductory experiment, a simple LED control circuit can be implemented. This helps in understanding the basic FPGA design flow, including writing Verilog code, assigning FPGA pins, compiling the design, and programming the board.

Pin Planning

For the push button, the following pin is used:

- **Pin 125:** Connected to i1

For the LEDs, the following pin assignments are used:

- **o[2] → Pin 134**
- **o[1] → Pin 135**
- **o[0] → Pin 133**

Verilog Code Example

```
module led_blink(input i1, output [2:0]o);

not n1(o[2],i1);
assign o[1] = i1;
not n2(o[0],i1);

endmodule
```

- The above Verilog code describes a simple **combinational circuit** containing two NOT gates and one direct connection. It has one 1-bit input, i1, and three output signals grouped together as the 3-bit vector o. When the input changes, the outputs change accordingly. The example uses both **gate-level modeling** and **data-flow modeling** styles.

Synthesis and Implementation

- After entering the Verilog code, the design is compiled using Quartus II. During this process, Quartus II synthesizes the HDL code into a hardware representation and then performs placement and routing on the FPGA. Once compilation is completed successfully, the required programming file is generated.

Programming the FPGA

- The generated programming file can then be loaded onto the FPGA using the **Quartus II Programmer** and a suitable programming cable. This configures the FPGA so that it performs the function described by the Verilog design.

Verification and Debugging

- Finally, the design should be tested on the actual hardware. Tools such as the **SignalTap II Logic Analyzer**, an oscilloscope, or a logic analyzer can be used to observe signals and verify the circuit's operation. The LED outputs can also be checked directly to confirm that they respond correctly to changes in the input.

4. Output

- After successfully programming and testing the FPGA, the expected output can be observed through the LEDs connected to the assigned FPGA pins. This confirms that the Verilog design has been correctly synthesized, implemented, programmed, and executed on the Cyclone II FPGA.